

Q1 :- What is the key difference between publishing a dashboard on Tableau Public versus Tableau Server/ Cloud in terms of accessibility and data security?

Answer :- The choice between **Tableau Public** and **Tableau Server/Cloud** represents a fundamental divide between "open data" and "enterprise security." While they share the same core visualization engine, their architectures for accessibility and protection are polar opposites.

1. Accessibility: Public vs. Private

The primary differentiator is **who** can see your work and **how** they find it.

- **Tableau Public:** This is a social platform for data enthusiasts. Once you publish, your dashboard is **indexed by search engines** (like Google) and becomes part of a searchable global gallery. It is essentially a "web page" that anyone with the link can view.
- **Tableau Server/Cloud:** These are gated environments. Accessibility is strictly controlled via **Authentication** (Logins/SSO). Even with a direct URL, an unauthorized person cannot view the dashboard; they will be met with a login screen.

2. Data Security and Privacy

This is the "deal-breaker" category for corporate or sensitive data.

Feature	Tableau Public	Tableau Server / Cloud
Data Visibility	Public. The underlying data is often accessible/downloadable by others.	Private. Data is encrypted and restricted to authorized users only.
Row-Level Security	Not available. Everyone sees the same data.	Robust. You can restrict data so User A sees Region A, and User B sees Region B.
Data Sources	Limited to static files (Excel, CSV) or Google Sheets.	Can connect to live databases (SQL, Snowflake, etc.) securely.

"Hidden" Workbooks	You can "hide" a viz from your profile, but the link is still technically discoverable.	Workbooks are invisible to anyone without explicit permissions.
---------------------------	---	---

Q2 :- When publishing a Tableau dashboard, what happens to a live connection compared to an extract? How does this affect data refresh and performance?

Answer :- 1. How Publishing Changes the Connection

The behavior depends on how Tableau "talks" to your data once it leaves your local computer.

- **Live Connection:** Tableau Server/Cloud acts as a pass-through. Every time a user interacts with the dashboard (filters, drills down, etc.), the Server sends a fresh SQL query to your original database.
 - *Requirement:* The Server must have the correct drivers and credentials (embedded or prompted) to reach your database.
- **Extract:** You are uploading a **.hyper** file (a compressed snapshot). The Server queries this local file instead of your database. The connection to the original database is essentially "severed" until a refresh is triggered.

2. Impact on Data Refresh

"Freshness" is the biggest trade-off here.

Feature	Live Connection	Extract
Update Frequency	Real-time. If a row is added to your SQL database, it appears the next time the dashboard is loaded/refreshed in the browser.	Snapshot. Data only updates when a "Refresh Task" runs. Between refreshes, the data is static.
Automation	Automatic by nature.	Requires a Refresh Schedule (e.g., daily at 8 AM) or a manual "Run Now" command on the Server.

Complexity	Simple for cloud databases (Snowflake, BigQuery). For on-premise data to Tableau Cloud, you must use Tableau Bridge .	Can be "Full" (replaces everything) or "Incremental" (only adds new rows based on a ID or Date column).
-------------------	--	---

3. Impact on Performance

This is where the user experience varies significantly.

Live Connection Performance

- **Variable:** Your dashboard is only as fast as your database. If your SQL Server is busy or unoptimized, your dashboard will be slow.
+1
- **Load:** High. Every single user interaction puts stress on your production database.
- **Best For:** Fast data warehouses (Snowflake, Redshift) where real-time accuracy is critical.

Q3 :- In Tableau Server or Cloud (conceptually), which feature allows administrators to control which users can view, interact with, or edit a published dashboard?

Answer :- 1. Site Roles (The "Badge")

Before a user can touch a dashboard, they must be assigned a **Site Role**. This is the maximum level of access a user can ever have.

- **Creator:** Can publish, edit, and manage content.
- **Explorer:** Can interact with and "Web Edit" existing dashboards but cannot publish new data sources.
- **Viewer:** Can view and interact (filters/sorting) but cannot edit or save changes.

2. Permission Capabilities (The "Keys")

Administrators assign specific **Capabilities** to users or groups for each piece of content. These are often grouped into templates like *View*, *Explore*, or *Publish*.

+1

- **View:** Can the user see the dashboard at all?
- **Interact:** Can they use filters, view tooltips, or add comments?

- **Edit:** Can they click the "Edit" button to change the layout or logic in the browser (Web Authoring)?
- **Download:** Can they take the data or the workbook file offline?

3. Projects (The "Filing Cabinet")

Rather than setting permissions for 100 individual dashboards, admins use **Projects**.

- **Locked Permissions:** You can "lock" a project so every dashboard inside it automatically inherits the same security settings. This is the best practice for maintaining consistency across a department.

Q4 :- If you want users to view a dashboard but not download the underlying data, which permission setting would you adjust in Tableau Server?

Answer :- The Specific Permissions to Toggle

To keep your data secure while allowing users to interact with the dashboard, you should focus on these two settings:

1. **Download Summary Data: * What it does:** Allows users to export the aggregated data currently shown in a specific chart (e.g., the total sales per region shown in a bar chart).
 - **Action:** Set this to **Denied** (the red "X") for the user or group.
2. **Download Full Data: * What it does:** This is the most sensitive setting. It allows users to see every underlying row that makes up a mark (e.g., every individual transaction in that region).
 - **Action:** Set this to **Denied**.

Where to Find These Settings

1. Navigate to the **Workbook** or **Project** on Tableau Server/Cloud.
2. Click the **Permissions** tab.
3. Select the **Group** or **User** (e.g., "Sales Interns").
4. In the permissions grid, look under the **Interaction** or **Data** category.
5. Explicitly click the **Deny** button for "Download Summary Data" and "Download Full Data."

Important Distinction: Workbook vs. Data Source

If you have published the **Data Source** separately from the workbook, you must also check the permissions on the Data Source itself.

- If a user has **Download/Save As** permissions on the *Data Source*, they might still be able to connect to it from a fresh Tableau Desktop instance and see the rows, even if the *Workbook* restricts them.
- **Best Practice:** Deny "Download" at both the Workbook and Data Source levels to be 100% safe.

Q5 :- Name two methods by which a Tableau dashboard can be embedded into a web page or website.

Answer :- **1. Using the Embed Code (Easiest Method)**

This is the most straightforward "copy-paste" method. It doesn't require advanced programming knowledge and is ideal for blogs, CMS platforms (like SharePoint or WordPress), or simple HTML pages.

- **How it works:** When you view a dashboard on Tableau Public, Server, or Cloud, there is a **Share** button. Clicking this provides an HTML snippet (an `<script>` tag or an `<iframe>`).
- **Implementation:** You simply paste this code into the HTML source of your webpage.
- **Best For:** Internal wikis or quick embeds where you don't need complex interaction between the website and the dashboard.

2. Using the Tableau Embedding API (Professional Method)

For a more seamless and interactive experience, developers use the **Tableau Embedding API** (currently version 3.x). This is a JavaScript library that gives you much finer control over how the dashboard behaves within your app.

+1

- **How it works:** You link the Embedding API library in your web page's header and use JavaScript to initialize the visualization within a specific `<div>` container.
- **Capabilities:** Unlike the simple embed code, the API allows your website to "talk" to the dashboard. For example, a button on your website can trigger a filter inside the Tableau viz, or the dashboard can send data back to your application.
- **Best For:** Custom web applications, enterprise portals, and "Embedded Analytics" products.

Q6 :- List two best practices you should follow when preparing a dashboard for public sharing to ensure data security and privacy.

Answer :- 1. Use Data Aggregation and Anonymization

Never publish "row-level" data. If your dataset contains individual names, IDs, or specific transaction details, you must transform it before it leaves your local machine.

- **Aggregate the Data:** Instead of showing every single sale in your `Sales_Transactions` dataset, aggregate the data at a higher level (e.g., Total Sales by Month or Region) during the **Extract** process.
- **Remove PII (Personally Identifiable Information):** Explicitly hide or delete columns containing names, email addresses, phone numbers, or exact GPS coordinates.
- **The "Rule of 5":** A common data privacy standard is to ensure no single data point represents fewer than five individuals. If a filtered view shows only one person's data, it is no longer anonymous.

2. Disable "Download" and "View Data" Capabilities

Even if your dashboard looks "safe" on the surface, the underlying workbook often contains the full dataset you used to build it. You must manually lock these doors before sharing the link.

- **Turn Off "Download Workbook and Data":** In the Tableau Public settings for your viz, uncheck the box that allows viewers to download the workbook. If a user downloads your `.twbx` file, they can open it in Tableau Desktop and see every hidden column and row you didn't intend to share.
- **Disable "View Data" in Tooltips:** By default, users can click a mark and see the "Full Data" behind it. You should disable this command in the **Tooltip** settings of every sheet to prevent curious users from "scraping" your data row-by-row.

Q7 :- After publishing a dashboard, users report a "Data not found" error. What is the most common cause related to data source settings?

Answer :- 1. The Primary Culprit: "Prompt User" vs. "Embedded Password"

In the publishing dialog, there is a **Data Sources** section with an "Edit" link.

- **The Error:** If this is set to **Prompt User**, the Server expects every viewer to have their own database username and password. Most viewers won't have this, so the connection fails.

- **The Fix:** Change the authentication mode to **Embedded Password**. This allows Tableau Server to use the credentials you provided during development to "speak" to the database on behalf of the users.

2. Other Common Configuration Issues

If embedding the password doesn't fix it, check these three secondary causes:

Cause	Problem	Solution
Missing Drivers	You have the database driver on your laptop, but it isn't installed on the Tableau Server node .	Contact your IT Admin to install the specific driver (e.g., PostgreSQL or Oracle) on the Server.
Network/Firewall	The database is behind a firewall that allows your laptop to connect but blocks the Tableau Server's IP address .	Ask your Database Administrator to "whitelist" the Tableau Server/Cloud IP.
File Path Errors	You used a local file (like <code>C:\Users\Documents\data.xlsx</code>) instead of a Universal Naming Convention (UNC) path (like <code>\\Server\Shared\data.xlsx</code>).	Use a UNC path so the Server can find the file on the network, or publish it as an Extract .

How to verify this in your current projects:

Since you're working with the `Sales_Transactions` and `Student_Performance` datasets:

1. **If they are Excel/CSV:** Always publish them as an **Extract** unless they are stored on a shared network drive the Server can reach.
2. **If they are in SQL:** Go to the "Data Sources" tab on your Tableau Server, click the three dots (...) on your data source, and select **Edit Connection** to ensure the password is saved and the "Test Connection" button returns a success message.

Q8 :- When embedding a Tableau dashboard into a secure internal portal, what two additional security measures should be implemented to ensure only authorized users can access the embedded content?

Answer :- 1. Configure "Connected Apps" with JWT

This is the modern standard for secure embedding (replacing the older "Trusted Authentication" for most users).

- **How it works:** You create a **Connected App** in Tableau Server or Cloud, which generates a **Client ID** and a **Secret**. Your internal portal's backend uses these to generate a **JSON Web Token (JWT)**.
+1
- **The Security Benefit:** The JWT is signed and short-lived (usually valid for only 5 minutes). It proves to Tableau that the request is coming from your specific, trusted web server and identifies exactly which user is viewing the page. This prevents "session hijacking" because the token expires almost immediately after the dashboard loads.

2. Implement a Domain Allowlist (CORS)

Even if someone steals your embed code, this measure ensures the dashboard **will not load** on any unauthorized website.

- **How it works:** In the Tableau Site Settings, you explicitly list the specific domains (e.g., <https://portal.yourcompany.com>) where your visualizations are allowed to appear.
- **The Security Benefit:** This prevents **Clickjacking** and "unauthorized embedding." If a malicious actor tries to iframe your dashboard into a different site, Tableau will detect that the request is coming from a domain not on your allowlist and block the content from rendering.